

Project 3: Minesweeper

Minesweeper Rules

COP3503C: Programming Fundamentals II
University of Florida

Fatemah Tavassoli, Nathan Bailey
Reviewed by Alex Johnson

Summer 2025 – Revision 1

Welcome to the third and final project of the course: Minesweeper. Minesweeper is a classic game, around since the 1960s. In this project, you will recreate Minesweeper from scratch, using only the c++ standard library and SFML.

Contents

1 Helpful Classes and Functions..... 3
1.1 Creating Windows..... 3
1.2 Welcome Window..... 3
1.3 Game Window..... 4
1.3.1 Buttons..... 4
1.3.2 Mouse Interactions..... 5
1.3.3 Adjacent Mines and Tiles..... 5
1.4 Leaderboard Window..... 5
2 Implementation Tips..... 5
2.1 Storing Resources..... 5
2.2 Managing Textures..... 6
2.3 Code Structure..... 6

1 Helpful Classes and Functions

1.1 Creating Windows

In this project, you will be working on three different windows: the welcome window, the game window, and the leaderboard window. A window in SFML is a graphical window that displays graphics and allows user interaction in a computer program.

We use the sf::RenderWindow to create a new object of the type render window. It is a subclass of sf::Window which you can draw images and text to. Here are some methods of the sf::RenderWindow class you will need:

Method	Use
RenderWindow(VideoMode mode, const String& title, std::uint32_t style, State state, const ContextSettings& settings)	Constructs a sf::RenderWindow, which, when executed, will also open a corresponding window on your screen.
bool isOpen()	Used to determine whether or not to continue running an event polling/drawing loop for an active window. Usually, this is found as the condition within a *while* loop after you have created your sf::RenderWindow.
std::optional<Event> pollEvent()	Returns and removes the first event from the window's event queue, or std::nullopt when the queue if empty. Events are added when your operating system detects an event, e.g.,you click your mouse, and removed when you call this function. You will need this function within a loop as the basis to interact with the keyboard and mouse.
void clear (Color color)	Clears the window before a new frame. You will need to run this *before* any draw() calls.
void draw(const Drawable& drawable, const RenderStates &states)	A method which will allow you to draw a sf::Drawable object onto the current frame. This includes both sf::Sprite and sf::Text objects.
void display()	Displays your newly-drawn frame to the screen. You should run this *after* all of your draw() calls.

Table 1.1: Useful sf::RenderWindow methods

Here's how you can create a simple sf::RenderWindow of size 800x600 pixels with the title "SFML Window" and the resize option disabled.

```
sf::RenderWindow window({sf::VideoMode({800, 600}), "SFML window", sf::Style::Close);
```

1.2 Welcome Window

Here are some helpful classes and functions you can use in this step:

Class/Function	Description
sf::Font	Loads and manages fonts for text rendering.
sf::Text	Displays and styles text (position, size, font, style).

sf::FloatRect	Defines a rectangle with floating-point dimensions, useful for positioning and bounding box calculations.
sf::Color	Manages RGBA colors for graphics elements.
sf::Vector2f	Represents a 2D vector, used for positions and sizes.
sf::Keyboard	Handles keyboard input, like key presses.
std::tolower()	Converts characters to lower case.
std::toupper()	Converts characters to upper case.
std::isalpha()	Checks if a character is alphabetic.
sf::Event	Represents and handles user/system events (e.g., key presses, mouse clicks).

Table 1.2: List of helpful classes and functions for the welcome window.

1.3 Game Window

Here are some of the concepts you will be using for this window:

Class/Function	Description
std::chrono	It is a part of the C++ standard library that provides support for date and time manipulation, including clocks, timers, time points, and durations. You will use this for implementing the timer functionality. • chrono::high_resolution_clock::now() is a C++ function that returns the current time point of the high-resolution clock. • chrono::duration_cast is a C++ function that converts a duration value from one unit to another, using the rules of the specified duration type. It allows you to convert between different time units, such as seconds, milliseconds, and microseconds, with precision and type safety.
sf::Texture	It is an object that represents an image loaded into memory and allows you to create sprites that display the image. It is a fundamental class for working with graphics in SFML.
sf::Sprite	It is a 2D object that displays a texture or a portion of a texture on the screen. It allows you to manipulate the position, scale, and rotation of the texture and is commonly used to display game characters and objects.
sf::IntRect	It is a simple rectangle class that represents a rectangular area of pixels, defined by its top-left corner and its width and height in pixels. It is often used to specify a portion of a texture that should be displayed by a sprite.
sf::Vector2f	It is a class in SFML that represents a 2D vector with floating-point coordinates.
sf::Mouse	It is a static class that provides access to the current state of the mouse, such as its position, buttons, and wheel and is used to handle mouse input.
sf::Event	It is an SFML data structure that represents user or system events, like mouse clicks or key presses, and enables you to handle them in your program. You will be using events like "sf::Event::Closed" and "sf::Event::MouseButtonPressed".

Table 1.3: Related concepts for the game window.

1.3.1 Buttons

A button is just an image that you can click on to make something happen. A more complex UI system would use an event/messaging system, but on a basic level you just need a sf::Sprite to represent the button, and every time the player clicks on something, you need to check if that mouse click occurred inside the boundaries of the sf::Sprite you're using as the button.
If you're drawing a sprite somewhere, you know its position (it's 0, 0 by default, or whatever you set it

to). You can get the width/height of the sprite through its textureRect, and then it's just a matter of checking if the mouse position is inside that box.

1.3.2 Mouse Interactions

The application can do "nothing" until the user clicks their mouse. Typically, games and many other applications clear/redraw the screen regularly (often dozens of times a second). Until the player clicks the mouse somewhere in the window, the program will appear to just sit there, idle. You won't see anything moving other than the timer.
Once the player has clicked, however (you can check for this in the event loop), you then need to do some checks about that click.

1.3.3 Adjacent Mines and Tiles

To calculate the number of nearby mines, as well as when revealing tiles, each tile should store a list of neighboring tiles. A tile could have UP TO 8 neighbors. An easy way to do this is with pointers. Since the number is a variable, a dynamically sized container would be perfect for this. You could also use a fixed-length array since no tile will ever have more than 8 neighbors.

- vector<Tile*> adjacent tiles; // Store each tile near us, the size() of each vector will vary.
- Tile* neighbors[8]; // Always 8 pointers, some of which might be nullptr.

1.4 Leaderboard Window

Here are some of the concepts you will be using for this window:

Class/Function	Description
sf::Font	You will need to load a font before you can display text.
sf::Text	This will be used to draw text onto the screen. It will allow you to change a lot of things like the text position, size, style (bold/underlined/italic), etc.
sf::Color	It is a simple class that represents an RGBA color. It allows you to define colors for drawing shapes, sprites, and text in your SFML application. You will mainly use this to set the text and background color.
sf::FloatRect	It is a class in SFML that represents a rectangular area with floating-point coordinates, defined by its top-left corner and its width and height in pixels. You can use this to store the local bounds of a sf::Text, which is helpful in setting the text to the center.
sf::Vector2f	It is a class in SFML that represents a 2D vector with floating-point coordinates.
sf::Event	It is an SFML data structure that represents user or system events, like mouse clicks or key presses, and enables you to handle them in your program. You will be using events like "sf::Event::Closed".
File I/O	In C++, file I/O operations are performed using the standard library classes and functions, such as std::fstream and std::ifstream for reading from files, and std::ofstream for writing to files.
std::ostringstream	Allows you to read and write formatted data to and from strings.
std::string::substr	substr() is a member function of the std::string class that allows you to extract a substring from a string, defined by a starting index and a length or by a starting index and an ending index.

Table 1.4: SFML concepts you will be using for the Leaderboard Window.

2 Implementation Tips

2.1 Storing Resources

While a program is running, it needs RESOURCES to get the job done—things like icons, textures, sound files, etc. Many of the resources need to be stored for long-term use, as they may be called upon time and time again... but you don't always know when they'll be needed when you compile your code.

2.2 Managing Textures

A great storage container for assets that you want to reference by name is the map<>. Storing something that you can access by its name with a container["NameOfAsset"] is vastly preferable to that of dealing with arrays—was "GameOver.png" stored in an array[25], or array[26]?

You may find it helpful to create a single storage container for all of the sf::Texture objects and then pass that around to any class which might need those files.

2.2 Managing Textures

Textures need to be kept alive in some sort of data structure to ensure that their memory isn't deleted and your program starts displaying white squares. You can store your textures in a map, where the name of the file will lead to the appropriate SFML texture. Listing 2.1 and Listing 2.2 are example implementations of this class.

```
#pragma once

#include <unordered_map>

#include <SFML/Graphics.hpp>



using namespace std;



class TextureManager {

   static std::unordered_map<string, sf::Texture> textures;



public:

   static sf::Texture& getTexture(string textureName);

};
```

Listing 2.1: TextureManager.h

```
#include <unordered_map>

#include "TextureManager.h"



unordered_map<string, sf::Texture> TextureManager::textures;



sf::Texture &TextureManager::getTexture(string textureName) {

    auto result = textures.find(textureName);

    if (result == textures.end()) {

        // Texture does not already exist in the map, go get it!

        sf::Texture newTexture;

        newTexture.loadFromFile("images/" + textureName + ".png");

        textures[textureName] = newTexture;

        return textures[textureName];

    } else {

        // Texture already exists, return it!

        return result->second;

    }

}
```

Listing 2.2: TextureManager.cpp

2.3 Code Structure

With larger programs, you can accomplish the goal in any number of ways. There isn't a single way to write this that works better than all others. From the outside perspective (i.e., that of a player), your application needs to DO various things. HOW you choose to accomplish those things is up to you. If you want to write a single, gigantic main() function, you are free to do so—that approach is not recommended, however. A few suggestions:

- A class to represent the board. This represents the core data object in the game.
- A class for tiles. The board is made up of a whole lot of these things. Each one of these can be a mine, have a flag, some number of adjacent tiles/mines, etc.

2.3 Code Structure

- You can have a separate Texture Manager class for loading textures and getting them at any point in time.

- Many programs (games or otherwise) do the same things over and over again while the application is running. The ability to easily (in code) reset everything is critical. Think about what sorts of helper functions you might want to make that happen. Things like:
  - Restarting the board.
  - Setting or clearing tiles of flags.
  - Setting or clearing mines (singly or in large quantities). – Recalculating the number of adjacent mines, etc.
- Separate classes for the implementation of the Welcome window and the Leaderboard window.